Lesson: Structures, Unions, and Enumerations

CSE 4107 : Structured Programming I

Shahriar Ivan

Structure Variables

- The properties of a ***structure*** are different from those of an array
  - The elements of a structure (its ***members***) aren't required to have the same type
  - The members of a structure have names; to select a particular member, we specify its name, not its position
- In some languages, structures are called ***records,*** and members are known as ***fields***
- A structure is a logical choice for storing a collection of related data items

Structure Variables

Declaring Structure Variables

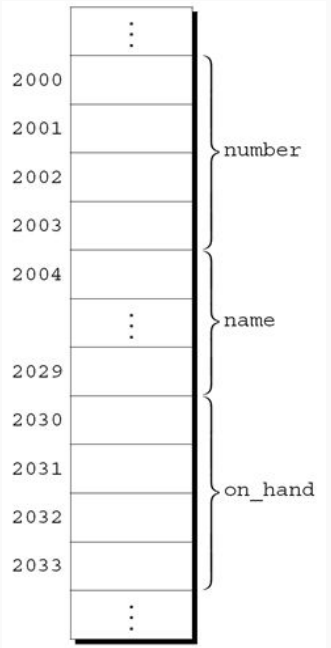
- A declaration of two structure variables that store information about parts in a warehouse:

```
struct {
    int number;
    char name[NAME_LEN+1];
    int on_hand;
} part1, part2;
```

Structure Variables

Declaring Structure Variables

- The members of a structure are stored in memory in the order in which they're declared
- Appearance of `part1`
- Assumptions:
  - `part1` is located at address 2000
  - Integers occupy four bytes
  - NAME_LEN has the value 25
  - There are no gaps between the members



Structure Variables

Declaring Structure Variables

- Each structure represents a new space ⇒ Name space
- Any names declared in that scope won't conflict with other names in a program

```
struct {
        int number;
        char name[NAME_LEN+1];
        int on_hand;
  } part1, part2;
```

```
struct {
        int number;
        char name[NAME_LEN+1];
        char gender;
  } employee1, employee2;
```

Structure Variables

Initializing Structure Variables

- With declaration ⇒ Similar to array initializer

```
struct {
    int number;
    char name[NAME_LEN+1];
    int on_hand;
}   part1 = {328, “North Hall”, 2},
    part2 = {329, “North Hall”, 1};
```

Structure Variables

Initializing Structure Variables

- Structure initializers follow rules similar to those for array initializers
- Expressions used in a structure initializer must be constant. (This restriction is relaxed in C99.)
- An initializer can have fewer members than the structure it’s initializing
- Any **“leftover”** members are given 0 as their initial value

Structure Variables

Designated Initializers (C99)

- Similar to arrays:

  `{.number = 328, .name = ”North Hall”, .on_hand = 2}`

- .member_name ⇒ Designator
- Easy to read and check for correctness
- Can be in a different order than the original declaration
  - Don’t have to remember the order
  - Declaration can be changed without affecting the initialization
  - Can be a mixed declaration: `{.number = 328, ”North Hall”, .on_hand = 2}`

Structure Variables

Operations on Structures

- To access a member within a structure, we write the name of the structure first, then a period, then the name of the member.
- Statements that display the values of part1's members:

```
printf("Part number: %d\n", part1.number);
printf("Part name: %s\n", part1.name);
printf("Quantity on hand: %d\n", part1.on_hand);
```

Structure Variables

Operations on Structures

- The members of a structure are *lvalues*
- They can appear on the left side of an assignment or as the operand in an increment or decrement expression:

```
part1.number = 258;          /* changes part1's part number */
part1.on_hand++;             /* increments part1's quantity on hand */
```

- The period used to access a structure member is actually a C operator
- It takes precedence over nearly all other operators

Structure Variables

Operations on Structures

- The = operator can be used only with structures of ***compatible*** types
- Two structures declared at the same time (as `part1` and `part2` were) are compatible
- Structures declared using the same *“structure tag”* or the same type name are also compatible
- Other than assignment, C provides no operations on entire structures
- In particular, the == and != operators can’t be used with structures

Structure Variables

Operations on Structures

- Arrays can't be copied using the = operator, but an array embedded within a structure is copied when the enclosing structure is copied.
- Some programmers exploit this property by creating *"dummy"* structures to enclose arrays that will be copied later:

```
struct { int a[10]; } a1, a2;
a1 = a2;                    /* legal, since a1 and a2 are structures */
```

Structure Types

- Suppose that a program needs to declare several structure variables with identical members
- We need a name that represents a *type* of structure, not a particular structure *variable*
- Ways to name a structure:
  - Declare a *“structure tag”*
  - Use typedef to define a type name

Structure Types

Declaring a Structure Tag

- A ***structure tag*** is a name used to identify a particular kind of structure
- The declaration of a structure tag named part:

```
struct part {
        int number;
        char name[NAME_LEN+1];
        int on_hand;
};
```

- Note that a semicolon must follow the right brace

Structure Types

Declaring a Structure Tag

- The part tag can be used to declare variables:

```
struct part part1, part2;
```

- We can't drop the word struct:

```
part part1, part2;          /*** WRONG ***/
```

- part isn't a *type name*; without the word struct, it is meaningless
- Since *structure* tags aren't recognized unless preceded by the word struct, they don't conflict with other names used in a program

Structure Types

Structures as Arguments and Return Values

- As arguments:

```
void print_name(struct part p){
        printf("Name: %s\n", p.name);
}
```

- As return value:

```
struct part build(int number, const char *name, int on_hand){
        struct part p;
        p.number = number;
        strcpy(p.name, name);
        p.on_hand = on_hand;
        return p;
}
```

Structure Types

Structures as Arguments and Return Values

- Passing a *structure* to a function and returning a *structure* from a function both require making a copy of all members in the *structure*
- To avoid this overhead, it's sometimes advisable to pass a pointer to a *structure* or return a pointer to a *structure*
- A call of build:

```
part1 = build(328, "North Hall", 1);
```

- A compound literal can be used to create a structure :

```
print_name((struct part) {528, "Disk drive", 10});
```

Nested Arrays and Structures

Nested Structures

- Structures and Arrays can be combined without any restrictions

```
struct person_name{
        char first[FIRST_NAME_LEN+1];
        char middle_initial;
        char last[LAST_NAME_LEN+1];
};
```

```
struct student{
        struct person_name name;
        int id,age;
        char gender;
} student1, student2;
```

Nested Arrays and Structures

Nested Structures

- To access:

```
strcpy(student1.name.first, "Hermione")
```

- Easier to treat name as units of data
- Can be passed to a function: `display_name(student1.name)`
- Copying information requires less assignments

```
struct person_name new_name;
...
student1.name = new_name;
```

Nested Arrays and Structures

Arrays of Structures

- Can be used to create simple databases

```
struct part inventory[100]
```

- Accessing a part can be done using array subscripting

```
print_name(inventory[i])
```

- Accessing a member combines subscripting and selection

```
inventory[i].number = 431
```

- Accessing a single character can be done too

```
inventory[i].name[0] = '\0'
```

Nested Arrays and Structures

Initializing an Array of Structures

- Initializing an array of structures is done in much the same way as initializing a multidimensional array
- Each structure has its own brace-enclosed initializer; the array initializer wraps another set of braces around the structure initializers
- One reason for initializing an array of structures is that it contains information that won't change during program execution

Nested Arrays and Structures

Initializing an Array of Structures

```
struct dialing_code {
        char *country;
        int code;
};

const struct dialing_code country_codes[] ={
        {"Argentina",                   54},  {"Bangladesh",          880},
        {"Brazil",                      55},  {"Burma (Myanmar)",  95},
        {"China",                       86},  {"Colombia",             57},
        {"Congo, Dem. Rep. of",      243}, {"Egypt",                20},
        {"Ethiopia",                    251}, {"France",               33},
        {"Germany",                     49},   {"India",               91},
 };
```

Nested Arrays and Structures

Initializing an Array of Structures

- A declaration of the inventory array that uses a designated initializer to create a single part:

```
struct part inventory[100] =
  { [0].number = 528, [0].on_hand = 10,
    [0].name[0] = '\0'};
```

- The first two items in the initializer use two designators; the last item uses three

Programming Task

Maintaining a Parts Database

- The program illustrates how nested arrays and structures are used in practice
- The program tracks parts stored in a warehouse
- Information about the parts is stored in an array of structures
- Contents of each structure:
  - Part number
  - Name
  - Quantity

Unions

- A ***union,*** like a structure, consists of one or more members, possibly of different types
- The compiler allocates only enough space for the largest of the members, which overlay each other within this space
- Assigning a new value to one member alters the values of the other members as well

Unions

- An example of a union variable:

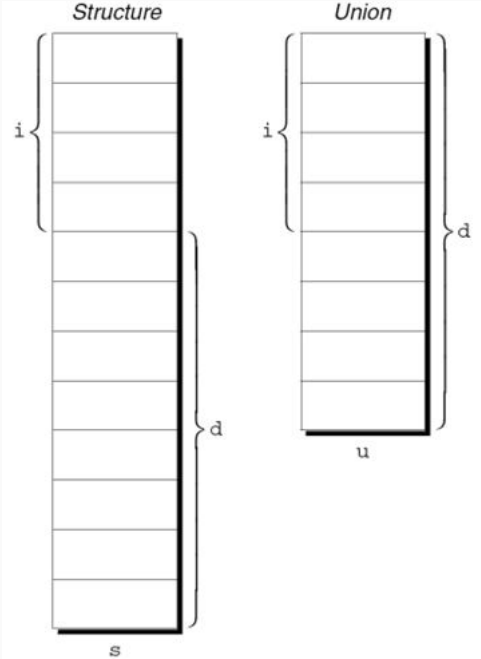
```
union {
    int i;
    double d;
} u;
```

- The declaration of a union closely resembles a structure declaration:

```
struct {
    int i;
    double d;
} s;
```

Unions

- The structure **s** and the union **u** differ in just one way
- The members of **s** are stored at different addresses in memory
- The members of **u** are stored at the same address



Unions

- Members of a union are accessed in the same way as members of a structure:

```
u.i = 82;
u.d = 74.8;
```

- Changing one member of a union alters any value previously stored in any of the other members
  - Storing a value in u.d causes any value previously stored in u.i to be lost
  - Changing u.i corrupts u.d

Unions

- The properties of unions are almost identical to the properties of structures
- We can declare union tags and union types in the same way we declare structure tags and types
- Like structures, unions can be copied using the = operator, passed to functions, and returned by functions
- Only the first member of a union can be given an initial value

Unions

- Initializing the i member of u to 0:

```
union {
        int i;
        double d;
} u = {0};
```

- Designated initializers can also be used with unions
- A designated initializer allows us to specify which member of a union should be initialized:

```
union {
        int i;
        double d;
} u = {.d = 10.0};
```

Unions

Using Unions to Save Space

- Unions can be used to save space in structures
- Suppose that we're designing a structure that will contain information about an item that's sold through a gift catalog
- Each item has a stock number and a price, as well as other information that depends on the type of the item:

Books: Title, author, number of pages

Mugs: Design

Shirts: Design, colors available, sizes available

Unions

Using Unions to Save Space

- A first attempt at designing the catalog_item structure:

```
struct catalog_item {
    int stock_number;
    double price;
    int item_type;
    char title[TITLE_LEN+1];
    char author[AUTHOR_LEN+1];
    int num_pages;
    char design[DESIGN_LEN+1];
    int colors;
    int sizes;  };
```

Unions

Using Unions to Save Space

- The item_type member would have one of the values BOOK, MUG, or SHIRT
- The colors and sizes members would store encoded combinations of colors and sizes
- This structure wastes space, since only part of the information in the structure is common to all items in the catalog
- By putting a union inside the catalog_item structure, we can reduce the space required by the structure

Unions

Using Unions to Save Space

```
struct catalog_item {
        int stock_number;
        double price;
        int item_type;
        union {
                struct {
                        char title[TITLE_LEN+1];
                        char author[AUTHOR_LEN+1];
                        int num_pages;
                } book;
                struct {
                        char design[DESIGN_LEN+1];
                } mug;
                struct {
                        char design[DESIGN_LEN+1];
                        int colors;
                        int sizes;
                } shirt;
        } item;
};
```

Unions

Using Unions to Save Space

- The catalog_item structure can be used to illustrate an interesting aspect of unions
- Normally, it's not a good idea to store a value into one member of a union and then access the data through a different member
- However, a ***special case***, when: two or more of the members of the union are structures, and the structures begin with one or more matching members
- If one of the structures is currently valid, then the matching members in the other structures will also be valid

Unions

Using Unions to Save Space

- The union embedded in the catalog_item structure contains three structures as members
- Two of these (mug and shirt) begin with a matching member (design)
- Now, suppose that we assign a value to one of the design members:

```
strcpy(c.item.mug.design, "Cats");
```

- The design member in the other structure will be defined and have the same value:

```
printf("%s", c.item.shirt.design);          /* prints "Cats" */
```

Unions

Using Unions to Build Mixed Data Structures

- Array with mixed data type

```
// Represents the kinds of data stored
typedef union{
        int i;
        double d;
} Number;
// Declaration
Number number_array[1000];
// Storing
number_array[0].i = 5;
number_array[1].d = 8.462;
```

Unions

Using Unions to Build Mixed Data Structures

- No easy way to tell which member of union was last changed
- Tags/Discriminants ⇒ Used to keep track of member

```
#define INT_KIND 0
#define DOUBLE_KIND 1
typedef struct{
        int kind;
        union{
                int i;
                double d;
        } u;
} Number;
```



```
Number n;
n.kind = INT_KIND;
n.u.i = 82;
…
if(n.kind == INT_KIND){
        printf("%d", n.u.i);
else
        printf("%f", n.u.d);
```

Enumerations

- In many programs, we'll need variables that have meaningful values
- A variable that stores the suit of a playing card should have only four potential values: "clubs," "diamonds," "hearts," and "spades."
- A "suit" variable can be declared as an integer:

```
int s;          /* s will store a suit */
s = 2;          /* 2 represents "hearts" */
```

- Problems with this technique:
  - We can't tell that s has only four possible values
  - The significance of 2 isn't apparent

Enumerations

- Using macros to define a suit “type” and names for the various suits is a step in the right direction:

```
#define SUIT        int
#define CLUBS       0
#define DIAMONDS    1
#define HEARTS      2
#define SPADES      3
```

- An updated version of the previous example:

```
SUIT s;
...
s = HEARTS;
```

Enumerations

- Problems with this technique:
  - There's no indication to someone reading the program that the macros represent values of the same "type."
  - If the number of possible values is more than a few, defining a separate macro for each will be tedious
  - The names CLUBS, DIAMONDS, HEARTS, and SPADES will be removed by the preprocessor, so they won't be available during debugging

Enumerations

- Enumerated type ⇒ Values are listed by the programmers
- Each value has a name ⇒ Enumeration constant
- Can be declared as a tag or by using typedef

```
enum suit {CLUBS, DIAMONDS, HEARTS, SPADES}; // tag
typedef enum {RED, GREEN, BLUE} Colors;
...
enum suit s1, s2;
Colors s1, s2
```

- Similar to #define, but not same ⇒ Has a scope